

2-Link Robot Arm Game

Development plan for turning the current slider-based simulation into a timed learning game

Project goal

Build an interactive Python game around the existing 2-link planar robot-arm simulator. The player controls the shoulder and elbow angles with sliders and must move the end effector onto a displayed target before time runs out. The game combines manual forward-kinematics control with inverse-kinematics feedback after a failed round.

Current starting point

- A 2-link planar arm with link lengths $L1 = 120$ mm and $L2 = 100$ mm.
- `forward_kinematics(theta1, theta2, L1, L2)` returns the base, elbow, and end-effector positions.
- Matplotlib displays the arm, a red target marker, an end-effector marker, and two angle sliders.
- The current basic version is a good stable baseline to keep committed in GitHub before game features are added.

Core round flow

Step	What happens
1	A new target appears on the workspace.
2	The arm begins in a chosen starting pose and a countdown starts.
3	The player moves θ_1 and θ_2 using the sliders.
4	The program continuously measures the distance from the end effector to the target.
5A	If the end effector reaches the target tolerance before the timer ends, the player wins the round.
5B	If time expires, the player loses and the program reveals the inverse-kinematics solution when one exists.
5C	The player may press “Can’t reach” to claim the target is unreachable. The game checks whether that claim is correct.
6	The score/attempt displays update and the next round begins.

Win detection

The end effector should not need to land on the exact target coordinate. Sliders move in discrete increments, so the game should use a circular success zone around the target.

The distance test is:

```
distance_to_target = hypot(end_effector_x - target_x, end_effector_y - target_y)
```

A sensible first value is a target tolerance of 8 mm. The round is won when `distance_to_target` is less than or equal to that tolerance. This test belongs in the existing slider update function, immediately after forward kinematics calculates the new end-effector position.

Important: add a `round_active` flag before awarding points. The slider callback runs repeatedly; without that guard, one successful position could accidentally create several wins.

Scoring system

The game uses a streak score. A player's current score represents consecutive correct rounds, so a loss resets the active score but preserves the best score achieved in the session.

Variable	Purpose	On a win	On a loss
streak_score	Current run of consecutive wins	+1	Reset to 0
top_score	Best streak this session	Update if streak_score is higher	No change
attempts	Total completed rounds	+1	+1

Example: after eight consecutive wins, the display reads Score: 8, Top score: 8, Attempts: 8. If the next round is lost, it becomes Score: 0, Top score: 8, Attempts: 9.

On-screen interface

- Arm display: links, joints, orange end-effector marker, and red X target.
- Sliders: θ_1 and θ_2 in degrees, likely from -180 to 180 degrees initially.
- Timer text: shows the time remaining, for example "Time: 7.4 s".
- Status text: shows messages such as "Target reached!", "Time up — showing solution", or "Wrong: this target was reachable".
- Scoreboard: Score, Top score, and Attempts, positioned using axes-relative coordinates so it remains in the plot corner.
- Button: "Can't reach". It checks the player's claim against the actual reachability of the current target.

Reachability rules

For link lengths L_1 and L_2 , the ideal geometric reach range is:

$$|L_1 - L_2| \leq \text{target_distance} \leq L_1 + L_2$$

For the current arm, the reachable radial range is 20 mm to 220 mm from the base. A target farther than 220 mm is too far. A target closer than 20 mm is too close for these unequal link lengths to fold onto exactly.

This mathematical test is independent of the slider limits. If you later restrict the joints more tightly than -180 to 180 degrees, some geometrically reachable targets may become unreachable in the game, so the target generator must also respect those joint limits.

“Can’t reach” button

Actual target state	Player presses “Can’t reach”	Round result
Reachable	Yes	Incorrect claim: lose the round, reset streak, then show IK solution.
Unreachable	Yes	Correct claim: win the round and increase streak.
Reachable	No; timer expires	Lose the round and reveal IK slider positions.
Unreachable	No; timer expires	Lose the round and display why it is out of reach.

Do not attempt to solve inverse kinematics for an unreachable target. The game should give a clear message, such as “Out of reach: maximum distance is 220 mm”.

Random target strategy

First version — reachable targets only: generate random angles within the slider limits, use forward kinematics on those angles, and make the resulting end-effector coordinate the target. This guarantees that a valid solution exists within the same angle range.

Later difficulty version — mixed targets: deliberately generate some targets outside the reachable annulus. Mark each target internally with `target_is_reachable` so the “Can’t reach” button can be checked reliably.

Avoid choosing random x/y values without testing them; that creates accidental unreachable targets and makes game behaviour inconsistent.

Timer behaviour

- Use a Matplotlib timer that calls a function repeatedly, for example every 100 ms.
- Store a round start time and calculate remaining time from the current time.
- Update timer text and redraw the canvas during each timer tick.
- When time reaches zero, end the round exactly once and disable further win checks until the next round.
- Do not use `time.sleep()` for the countdown because it freezes the Matplotlib window and prevents slider interaction.

Inverse kinematics reveal

When a reachable round times out—or the player wrongly claims “Can’t reach”—the game should calculate a valid pair of joint angles and move both sliders to that solution. With your convention, θ_1 is the first-link angle and θ_2 is the relative elbow angle; forward kinematics uses $\theta_1 + \theta_2$ for the second-link’s absolute angle.

A 2-link arm normally has two possible valid poses: elbow-up and elbow-down. Start by implementing one consistently. Later, you can reveal both solutions or choose the one closest to the player’s current pose.

When setting sliders programmatically, make sure the round is inactive first. Slider value changes trigger the same update callback used by the player.

Recommended milestones

Milestone	Deliverable
0. Baseline	Commit the current slider-and-fixed-target version to GitHub.
1. Hit detection	Add tolerance and print a temporary success message when the tip reaches the target.
2. One-win guard	Add <code>round_active</code> so a single target can only be completed once.
3. Scoreboard	Add <code>streak_score</code> , <code>top_score</code> , <code>attempts</code> , and live plot text.
4. New target	After a win, generate and draw a new reachable target.
5. Timer	Add countdown text and a timeout result.
6. IK reveal	On a reachable timeout, compute solution angles and set both sliders.
7. Can't reach	Add button handling and mixed reachable/unreachable targets.
8. Polish	Adjust difficulty, target tolerance, colours, messages, README, and screenshots.

File responsibilities

- **kinematics.py**: keep reusable robotics maths here: forward kinematics now; inverse kinematics later.
- **main.py**: keep display, sliders, targets, timer, buttons, score state, and game-round logic here.
- **README.md**: explain installation, how to run, gameplay rules, and future improvements.
- **.gitignore**: exclude `.venv/`, `__pycache__`, and `*.pyc` from GitHub.

First implementation task

Begin with Milestone 1 only: add `TARGET_TOLERANCE = 8` near the link constants. In the slider update function, calculate the end-effector-to-target distance using `np.hypot`. When that distance is within tolerance, temporarily print "Target reached!". Test it manually before building scoring, randomisation, timers, or buttons.